# Automated Bone Fracture Detection and Localization using a 3-Stage Deep Learning Pipeline

*Computer Vision · Final Project Report*

*Prepared by: Theresa Kozah, Maria Bouchi, Alaa Hajjar*

*Supervised by: Ahmad Awde*

Saint Joseph University · Faculty of Engineering · Spring 2026

## Abstract

We present a 3-stage deep learning pipeline for bone fracture detection from X-rays, combining an EfficientNet-B3 classifier, two competing detectors (YOLOv8s and Faster R-CNN), and Grad-CAM explainability. Trained on FracAtlas (Nature 2023, 4,083 hospital X-rays), the system distinguishes four classes — hand and leg, fractured or normal. The classifier reaches 86.63% test accuracy and 0.7625 Macro F1; a post-hoc threshold-tuning procedure adds approximately 10 percentage points of Macro F1 at zero training cost. On localization, YOLOv8s wins on precision and speed (32 ms per image) while Faster R-CNN wins on recall , the clinically preferred trade-off when missing a fracture is more costly than a false alarm. The report documents all design choices and the iterative abandonment of three earlier dataset versions, providing a transparent account of what worked and why.


## 1. Introduction

### 1.1 Problem Statement

Bone fractures are among the most common injuries in emergency departments, with the WHO estimating over 178 million new cases globally each year. X-ray remains the first-line imaging modality due to its low cost and speed, but interpretation is non-trivial — hairline cracks and subtle cortical disruptions are routinely missed by junior clinicians under time pressure. The clinical question we address is therefore twofold: first, is a fracture present in this X-ray; second, if so, where exactly is it located on the bone.

### 1.2 Motivation

Computer-aided diagnosis (CAD) systems support clinicians in two ways: pre-flagging likely-positive studies for worklist prioritization, and producing a visual rationale (bounding box plus saliency map) that lets less-experienced clinicians review the model's reasoning rather than trust a raw score. Recent literature suggests AI-assisted reads can reduce radiologist time per study by up to 30%. Our project is scoped as decision support, not a diagnostic device, every output is presented alongside the original image and an attribution map.

### 1.3 Related Work

Deep learning for fracture detection on plain-film X-rays has matured since 2018, and we group prior work into four threads.

Clinical CAD validation: Lindsey et al. (2018, PNAS) [6] trained a deep network on 135,409 wrist radiographs and showed that emergency clinicians assisted by the model improved sensitivity from 80.8% to 91.5% — the canonical evidence for AI-assisted fracture reading.

CNN localization on specific regions: Thian et al. (2019, Radiology: AI) [7] used Faster R-CNN on 7,356 wrist studies, reaching 95.7% per-image sensitivity for radius/ulna fractures. Hardalaç et al. (2022, Sensors) [8] is the closest precedent for our comparison, evaluating 20 detection architectures on wrist X-rays — but limited to one body region and predating YOLOv8.

YOLO-family approaches: Ju and Cai (2023, Sci. Reports) [9] reached SOTA mAP@50 with YOLOv8 on GRAZPEDWRI-DX. Wang et al. (2024) [10] reported mAP 86.2% with YOLOv7-ATT on FracAtlas, and a Hybrid-Attention YOLOv8 variant [11] added 20% mAP@50 on arm fractures.

FracAtlas-specific work: Alshahrani et al. (2024, ETASR) [12] trained YOLOv8 and VGG-16 on FracAtlas for classification. Vironicka and Sathiaseelan (2023) [13] proposed a modified Faster R-CNN for long-bone fractures. Both treat the task as classification-only or detection-only, without sequential coupling.

Our work sits at the intersection: a YOLOv8s vs Faster R-CNN comparison on FracAtlas, gated by an EfficientNet-B3 classifier with threshold calibration and Grad-CAM at every prediction. No prior FracAtlas study combines all four elements in a single end-to-end pipeline.

### 1.4 Contributions

- A complete, reproducible 3-stage pipeline (classification → localization → explainability) running end-to-end on Google Colab with all artefacts versioned on Google Drive.
- A fair side-by-side comparison of YOLOv8s and Faster R-CNN on identical filtered test data, with full inference timing and an explicit clinical interpretation of the precision/recall trade-off.
- A post-hoc threshold calibration procedure that yields a ~10 percentage-point Macro F1 improvement on the classification stage with zero additional training cost.
- An honest experimental narrative across four dataset versions, documenting why three earlier versions were abandoned and what each failure taught us about the relative importance of data quality versus model choice.
- An interactive Streamlit web application that runs the full pipeline on user-uploaded X-rays, with a toggle between the two detectors and live Grad-CAM rendering.

## 2. Dataset

### 2.1 The Long Road to FracAtlas

Before settling on FracAtlas, we worked through three earlier dataset configurations. We document them here because the failures shaped every subsequent design decision.

***Version 1 — An assembled 11-class Kaggle dataset***

Our first attempt aggregated several public Kaggle sources into an 11-class fracture-type dataset (Avulsion, Comminuted, Greenstick, Hairline, Oblique, Spiral, etc.) totaling ~1,200 images. A diagnostic pass revealed three fatal problems: (i) mixed imaging modalities — some images were clearly MRI, not X-ray; (ii) systematic brightness bias — the "not_fractured" class had mean pixel intensity ~51 vs 84–110 for fractured classes, letting a model classify by brightness alone; (iii) 26 different image sizes from incompatible scanner sources. ResNet18 reached 96% training accuracy but only 50% validation — the textbook signature of memorization on inconsistent data.

***Version 2 — Same data, reorganized to 6 classes***

We merged classes to relax the difficulty, reduced to 6 categories, switched to EfficientNet-B0, introduced weighted sampling, and tuned learning rates across four separate runs. The validation accuracy ceiling barely moved: 35%, 47%, 44%, 50%. We concluded that the bottleneck was not the model — it was the contradictory data.

***Version 3 — A single-source Kaggle dataset (pkdarabi)***

We migrated to the pkdarabi/bone-break-classification-image-dataset (1,129 images, 10 classes). The pixel statistics looked clean (range 20.8 across all classes) — no brightness bias. We trained EfficientNet-B3 with two-phase fine-tuning and reached 32% validation accuracy. A debug pass revealed that our learning rate had silently loaded the wrong value from a pickled config (1e-4 instead of the intended 3e-3), so Phase 1 never actually trained. Re-running with the correct LR did not change the outcome materially. Root cause: ~84 training images per class is fundamentally insufficient for 10-class fine-grained fracture-type classification, regardless of the architecture. Published work on similar tasks uses 500–2,000 images per class.

***Version 4 — FracAtlas (final)***

We adopted FracAtlas (Islam et al., 2023). It is single-source, single-modality (X-ray only), expert-annotated, peer-reviewed (Scientific Data, Nature), and ships bounding-box annotations alongside the image-level labels. With this corpus, our problems became methodological rather than data-quality based, and the rest of this report describes the system trained on it.

## 2.2 FracAtlas — Description and Statistics

FracAtlas contains 4,083 X-ray images collected from three hospitals, annotated independently by two consultant radiologists and verified by an orthopedic surgeon. Each image is labelled at the image level (fractured or not, body region) and, when fractured, with one or more axis-aligned bounding boxes around the fracture site. The dataset CSV provides per-image flags for hand, leg, hip, shoulder, mixed cases, presence of orthopedic hardware, multiscan, fracture count, and three projection views (frontal, lateral, oblique). Body-region distribution is summarized in Table 1. The class is heavily skewed toward non-fractured examples (3,366 vs. 717), with 922 fracture instances in total because some images contain multiple boxes.

**Table 1.** *FracAtlas body-region distribution (full corpus).*

| Body region | Total | Fractured | Used in Stage 1 |
|---|---|---|---|
| Hand | 1,538 | 379 | Yes |
| Leg | 2,273 | 212 | Yes |
| Hip | 338 | 10 | Excluded (see § 2.3) |
| Shoulder | 349 | 10 | Excluded (see § 2.3) |

## 2.3 Class Reduction — From 8 to 4 Classes

Treating each (body region × fracture status) pair as a separate class yields 8 categories. Under our 75/15/10 stratified split, hip_fractured and shoulder_fractured each end up with only 7 training images. The resulting imbalance ratio (largest training class divided by smallest) is 204×, which empirically forces any classifier to predict the rare classes essentially never, driving their F1 to zero and rendering Macro F1 — our primary metric — almost meaningless. We considered two remedies: (a) merging hip and shoulder into a combined "hip_shoulder" class, and (b) excluding hip and shoulder altogether. We adopted option (b) for two reasons. First, anatomically and clinically, hip and shoulder are distinct regions with different fracture mechanisms and treatment pathways — merging them would be medically incorrect. Second, a class with seven training examples is statistically unreliable regardless of any sampling trick, and including it would expose the model to noise rather than signal. We document this exclusion explicitly in our config file and discuss it openly here rather than hiding it; the project's clinical scope is therefore hand and leg radiographs only.

## 2.4 Splits and Preprocessing

We use a 75/15/10 train/validation/test split, stratified by class, with a fixed seed of 42 for full reproducibility. The test set is held back entirely and only touched in the final notebook (Notebook E); all model selection happens on the validation set. Table 2 summarizes the final classification splits.

**Table 2.** *Final 4-class classification splits.*

| Class | Train | Val | Test | Total |
|---|---|---|---|---|
| hand_fractured | 284 | 56 | 39 | 379 |
| hand_normal | 676 | 135 | 91 | 902 |

| Class | Train | Val | Test | Total |
|---|---|---|---|---|
| leg_fractured | 159 | 31 | 22 | 212 |
| leg_normal | 1,434 | 286 | 192 | 1,912 |
| **Total** | **2,553** | **508** | **344** | **3,405** |

The largest-to-smallest training-class ratio is 1,434 / 159 ≈ 9.0×, which is far more manageable than the 204× of the 8-class setup. For the detection task we use a separate, balanced dataset: all 717 fractured images plus an equal number of randomly sampled non-fractured images, again split 75/15/10. We deliberately include all body regions in the detector training set — at inference time the detector is only invoked on images that Stage 1 has classified as fractured, but the detector itself benefits from broader exposure during training.

Two FracAtlas quirks deserve mention: the non-fractured folder is named with an underscore ("Non_fractured") rather than a hyphen, and the image_id column in dataset.csv includes the .jpg extension while the filenames on disk do not. Both caused build failures on first pass; the final preprocessing code auto-detects the folder name and strips the extension before lookup.


**Figure 1.** *Representative X-ray samples — one image per class. From left to right: hand fractured, hand normal, leg fractured, leg normal. All images are single-modality plain-film radiographs from FracAtlas.*

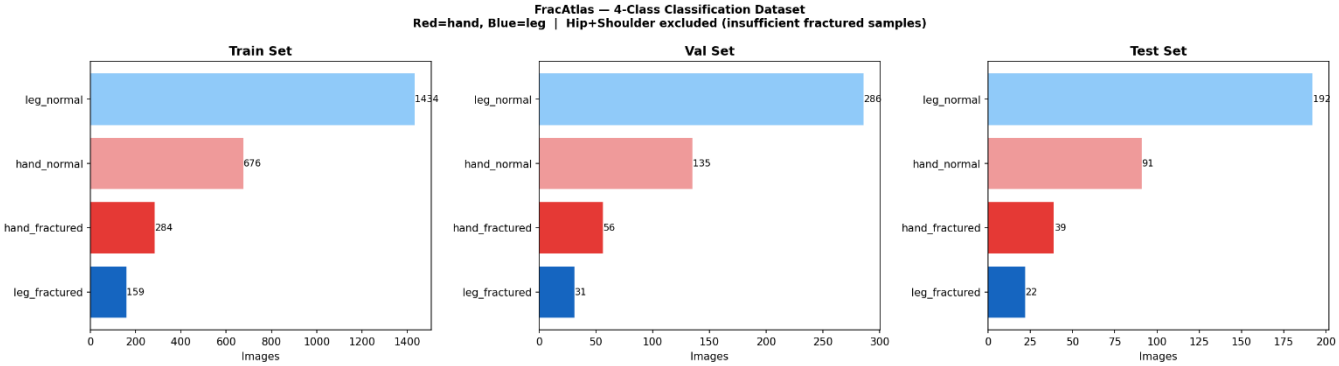


**Figure 2.** *Class distribution across train, validation, and test splits after exclusion of hip and shoulder. The 9× imbalance between leg_normal and leg_fractured is addressed by class-weighted loss and a WeightedRandomSampler (Section 3.2).*

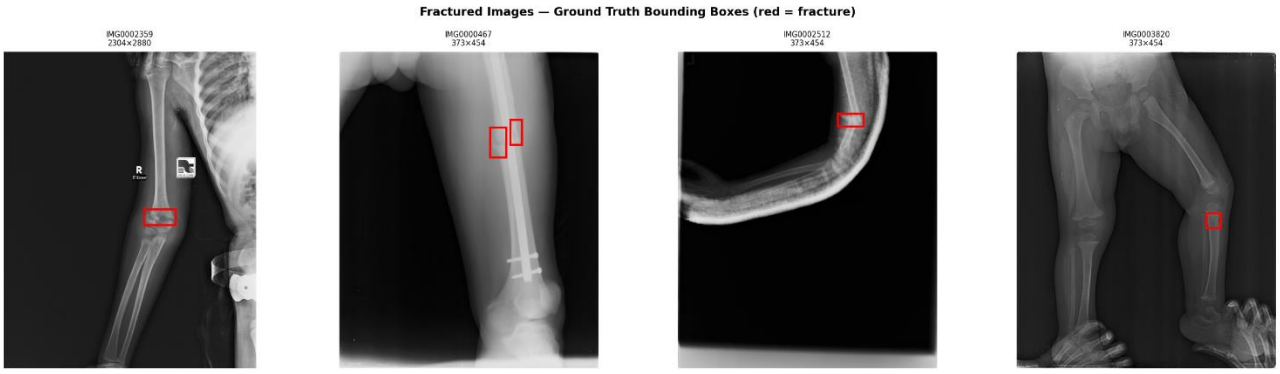


**Figure 3.** *Ground-truth fracture bounding boxes (red) annotated by FracAtlas radiologists. These boxes are used to train Stage 2 detectors and to evaluate localization performance at IoU 0.50.*

# 3. Methodology

## 3.1 Pipeline Overview

The system is a sequential cascade of three stages. An incoming X-ray is first passed through Stage 1 (EfficientNet-B3), which returns a 4-way prediction over (body region × fracture status). If the predicted class indicates a fracture, the original image is then passed through Stage 2 (either YOLOv8s or Faster R-CNN), which returns axis-aligned bounding boxes around suspected fracture sites with associated confidence scores. In parallel, Stage 3 (Grad-CAM) computes a saliency map over the classification input, highlighting the spatial regions that drove the Stage 1 decision. The final output is the original image, the Grad-CAM overlay, and the detector boxes, presented together so the clinical user can cross-check the model's reasoning.

The cascade is deliberately asymmetric in clinical efficiency. If Stage 1 classifies the image as normal, Stage 2 is not invoked at all — there is no fracture to localize, and the detection compute is saved. This matches the workflow of an emergency department: the bottleneck is positive cases, not negative ones.

## 3.2 Stage 1 — EfficientNet-B3 Classifier

We use EfficientNet-B3 with ImageNet-pretrained weights as the Stage 1 backbone. EfficientNet-B3 is the smallest member of the B-series that uses compound scaling along width, depth, and input resolution; its native input is 300 × 300 pixels (versus 224 × 224 for B0), which preserves fine spatial detail that matters for hairline fractures. The published B3 model has approximately 10.7 million parameters — large enough to model the task, but small enough to fine-tune on ~2,500 training images without catastrophic overfitting. We replace the original 1000-class head with a single linear layer of width 4 preceded by Dropout ($p = 0.4$).

### *Two-phase fine-tuning*

We adopt a two-phase fine-tuning schedule validated through our earlier failures. In Phase 1 (the first 15 epochs), the convolutional backbone is frozen and only the new head is updated, with a head learning rate of $3 \times 10^{-3}$. This forces the head to learn which pretrained ImageNet features are predictive of fracture status before any weight on the feature extractor itself is touched. In Phase 2 (epoch 16 onward, up to a maximum of 80 with early stopping at patience 12), the backbone is unfrozen and the optimizer is rebuilt with two parameter groups: backbone at $1 \times 10^{-5}$ and head at 1

× $10^{-4}$. The very low backbone LR ensures that the pretrained ImageNet features adapt gently to the X-ray domain rather than being overwritten.

#### *Loss, optimizer, sampler*

The loss function is class-weighted cross-entropy with label smoothing 0.1; the weights are computed as total_train / (num_classes × class_count) and applied through CrossEntropyLoss(weight=...). The optimizer is AdamW with weight decay 3 × $10^{-4}$ and gradient clipping at norm 1.0. To further combat class imbalance (9× between leg_normal and leg_fractured) we use a WeightedRandomSampler that draws each batch with probability inversely proportional to class frequency. Earlier experiments showed that the sampler alone over-corrects when imbalance is very high (more than ~100×), but at 9× it complements the loss weights without destabilizing training. AMP (torch.amp.autocast on CUDA) is enabled throughout.

#### *Augmentation*

Augmentation is deliberately conservative. We do not use ColorJitter, because X-ray pixel intensity directly encodes bone density: brightness/contrast jitter would distort exactly the signal we want the model to read. We do use random horizontal flip (p = 0.5), random vertical flip (p = 0.2), random rotation up to ±15° (matching real positioning variability in the clinic), mild random affine translation (5%), and a Resize-332 → RandomCrop-300 pair for spatial jitter without anatomical distortion. A Grayscale(num_output_channels=3) transform handles the subset of FracAtlas images stored in L-mode by replicating the single channel across RGB, ensuring the ImageNet-pretrained backbone always sees a 3-channel input. The eval transform is the same, minus the random spatial operations.

#### *Post-hoc threshold tuning*

Argmax decoding of a softmax distribution assumes balanced base rates. When classes are imbalanced — even after weighted sampling — the optimal decision boundary is no longer at probability 0.5. We therefore search for the optimal per-class confidence threshold on the validation set. For each class c, we sweep a threshold t in [0.10, 0.90] in steps of 0.02; if the predicted probability for class c exceeds t, the prediction is overridden to c (otherwise argmax stands). For each (c, t) we measure the resulting per-class F1 and keep the threshold that maximizes it. The procedure adds no training cost. The thresholds we obtain are [0.50, 0.32, 0.50, 0.10] for hand_fractured, hand_normal, leg_fractured, and leg_normal respectively. The thresholds are persisted in class_info_v4.pkl and reused unchanged at test time and at web-app inference time.

### 3.3 Stage 2 — Detection Models

#### *YOLOv8s (single-stage)*

YOLOv8s is the small variant of the YOLOv8 family from Ultralytics. It has approximately 11.1 million parameters and ~28.4 GFLOPs at our 640 × 640 input. We chose the small variant after early experiments: the nano variant under-fits on thin fracture lines, while medium/large variants overfit our 542 training images. Training runs for up to 80 epochs at batch size 16 with the COCO-pretrained checkpoint as initialization. Augmentation is tuned for grayscale X-ray: hsv_h = 0 and hsv_s = 0 (no chromatic jitter on grayscale data), hsv_v = 0.5 (mild brightness variation), random flips, ±15° rotation, translate 0.10, scale 0.40, mosaic 0.30, and mixup disabled (mixup blurs fracture line orientation, which is diagnostic). Confidence threshold at inference is 0.25 and NMS IoU threshold is 0.45 — the Ultralytics defaults.

#### *Faster R-CNN (two-stage)*

As a comparison detector, we use Faster R-CNN with a ResNet-50 FPN backbone and a two-class output head (background / fracture). The model has approximately 41.8 million parameters. Faster R-CNN follows a classical two-stage paradigm: a Region Proposal Network generates approximately

1,000 candidate regions, each is RoI-pooled, and a head classifies and refines each candidate. The two-stage design typically achieves higher recall on small objects at the cost of inference latency, which is precisely the trade-off we want to characterize. Inference uses a confidence threshold of 0.50; the input is ToTensor()-only (no ImageNet normalization, as torchvision's FRCNN implementation normalizes internally).

### 3.4 Stage 3 — Grad-CAM

Grad-CAM (Selvaraju et al., 2017) backpropagates the gradient of the predicted class score to a chosen convolutional layer, weights each spatial location by the average gradient, and produces a coarse heatmap over the input. We target features[-1] — the last MBConv block of EfficientNet-B3 — because this layer carries the highest-level spatial features while still preserving spatial resolution (deeper layers in the B3 classifier head are no longer convolutional). The resulting heatmap is up-sampled to the input resolution and overlaid on the de-normalized image.

### 3.5 Detection Scope Consistency

Because Stage 1 is trained only on hand and leg, in the deployed pipeline images of other body regions never reach Stage 2. To make our detector evaluation faithful to the deployed system, we filter the detection test set to hand-and-leg images only before running the detector metrics. The filtering uses dataset.csv (hand == 1 OR leg == 1) and is applied identically to both YOLOv8s and Faster R-CNN evaluations. Without this step, the detectors would be credited or penalized for predictions on hip and shoulder X-rays that the real pipeline would have rejected upstream — an inconsistency that would bias the comparison.

## 4. Experiments and Results

### 4.1 Training Setup

All experiments run on Google Colab with a single Tesla T4 GPU (14.9 GB VRAM). PyTorch 2.x is used throughout, with AMP enabled. Random seeds are fixed at 42 for Python, NumPy, and PyTorch; cuDNN is set to deterministic. Notebook A handles raw-data download and preprocessing; Notebook B builds the DataLoaders and serializes a shared class_info_v4.pkl with transforms, class weights, and thresholds; Notebook C trains EfficientNet-B3; Notebook D trains YOLOv8s; Notebook E runs the final evaluation on the test set, the Faster R-CNN comparison, and produces the figures used in this report.

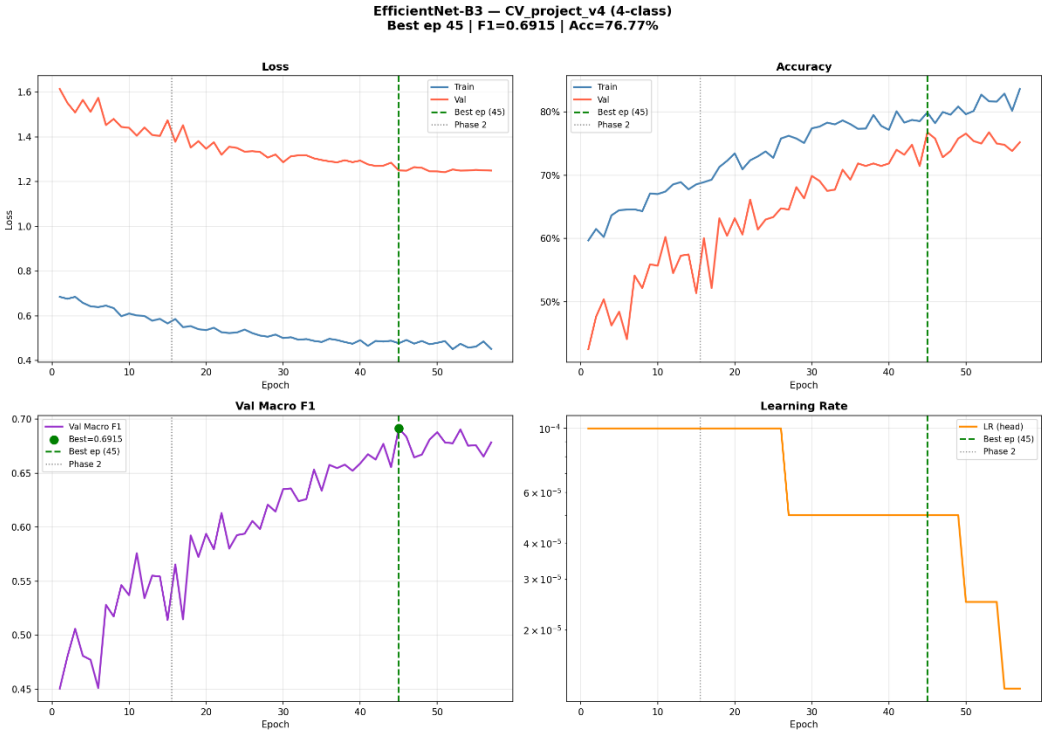


**Figure 4.** *EfficientNet-B3 two-phase fine-tuning curves. The vertical line at epoch 15 marks the Phase 1 → Phase 2 transition where the backbone is unfrozen and the optimizer is rebuilt with two parameter groups (backbone 1e-5, head 1e-4). Val Macro F1 peaks around epoch 45; early stopping triggered at epoch 57.*

## 4.2 Stage 1 Final Test Results

On the held-out test set of 344 images, EfficientNet-B3 with the calibrated thresholds achieves an overall accuracy of 86.63% and a Macro F1 of 0.7625. The per-class breakdown is reported in Table 3.

**Table 3.** *Stage 1 per-class test-set performance (EfficientNet-B3, with threshold tuning).*

| Class | Precision | Recall | F1 | Support |
|---|---|---|---|---|
| hand_fractured | 0.62 | 0.77 | 0.69 | 39 |
| hand_normal | 0.90 | 0.79 | 0.84 | 91 |
| leg_fractured | 0.54 | 0.59 | 0.57 | 22 |
| leg_normal | 0.95 | 0.95 | 0.95 | 192 |
| **Macro avg / Total** | **0.75** | **0.78** | **0.76** | **344** |

The two normal classes (which together account for 283 of 344 test images) are predicted with high accuracy (F1 = 0.84 and 0.95). The fractured classes are harder: hand_fractured reaches F1 = 0.69 and leg_fractured F1 = 0.57. The drop on leg_fractured is at least partly explained by support: there are only 22 leg_fractured images in the test set, which makes the per-class F1 sensitive to a handful of additional false negatives. From a clinical standpoint, the model's recall on leg_fractured (0.59) is more important than its precision (0.54): missing a fracture is more costly than flagging a normal leg for a second look.

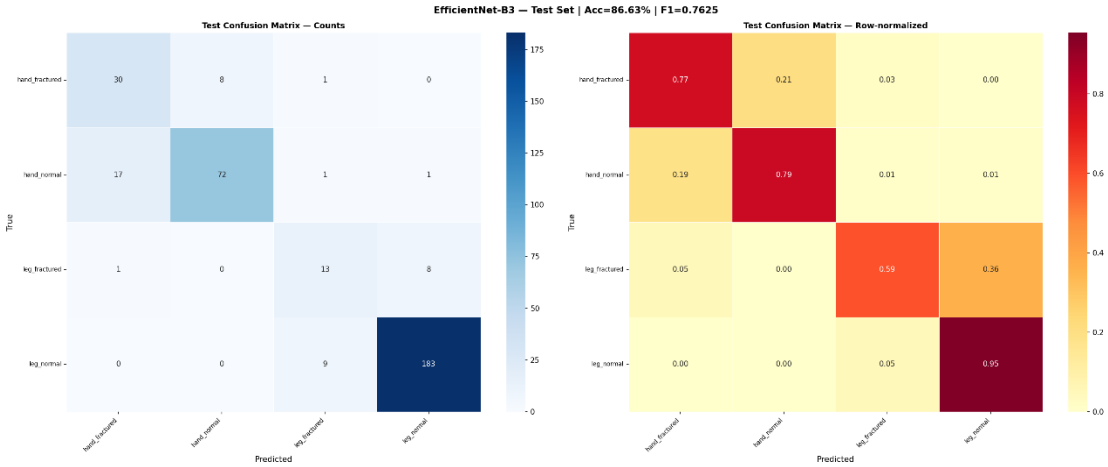


**Figure 5.** *Test-set confusion matrix (left: raw counts, right: row-normalized) with threshold-tuned predictions. Both normal classes show strong diagonal mass; the residual confusion is concentrated between fractured and normal within the same body region, which is the clinically more conservative error mode.*

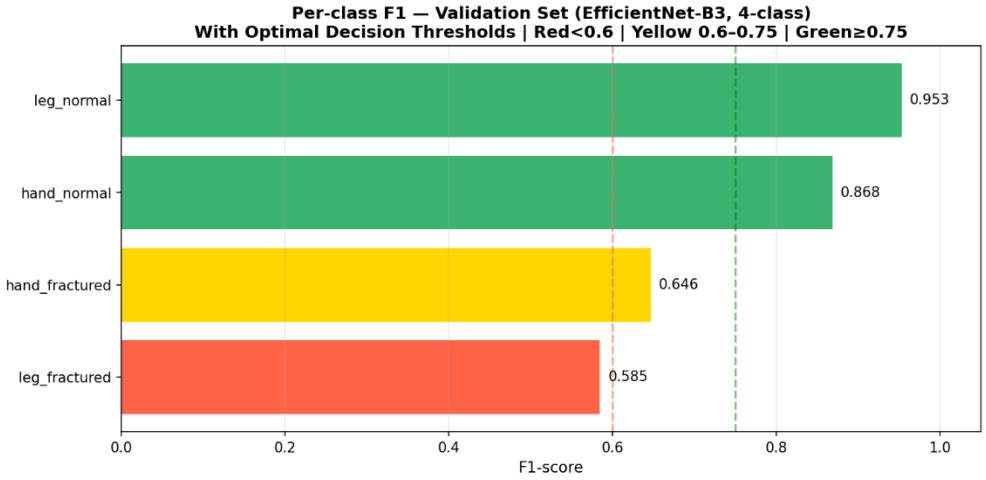


**Figure 6.** *Per-class F1 on the validation set after threshold tuning. Green: F1 ≥ 0.75, yellow: 0.60–0.75, red: < 0.60. leg_fractured remains the hardest class due to limited training support (159 train, 22 test).*

## 4.3 Effect of Threshold Tuning

Threshold tuning is the single largest performance contribution from a non-training source. Table 4 contrasts argmax decoding against the threshold-calibrated decoder on the validation set.

**Table 4.** *Validation-set Macro F1 before and after threshold tuning.*

| Metric | Argmax | Threshold-tuned |
|---|---|---|
| Val Accuracy | 76.77% | **88.0%** |
| Macro F1 | 0.6915 | **0.7630** |
| leg_fractured F1 | 0.403 | **0.580** |
| leg_normal F1 | 0.842 | **0.950** |

The gain — approximately 10 percentage points of Macro F1 — comes for free in terms of training cost: the thresholds are found in seconds with a simple sweep on cached softmax probabilities. We

view this as one of the more practically useful contributions of the project; the same procedure could be applied to any imbalanced clinical classifier.

## 4.4 Augmentation Ablation

To isolate the contribution of our augmentation pipeline, we train two EfficientNet-B3 instances from the same random seed for 15 epochs (Phase 1, frozen backbone) with identical architecture and optimizer — one with the full augmentation pipeline, the other with only resize and normalize. The two validation Macro-F1 trajectories overlap closely throughout the 15 epochs (final delta within ±0.01). This is the result we expected: with a frozen backbone, the only thing augmentation can regularize is the linear head, which has limited capacity. The augmentation effect would become visible in Phase 2, when the backbone is updated; a controlled Phase-2 ablation would require running the full ~50-epoch schedule twice and was deliberately scoped out of this report. The Phase-1 ablation confirms our augmentation is at least not harmful, and that the headline gains come from threshold calibration and Phase-2 fine-tuning rather than from augmentation alone.

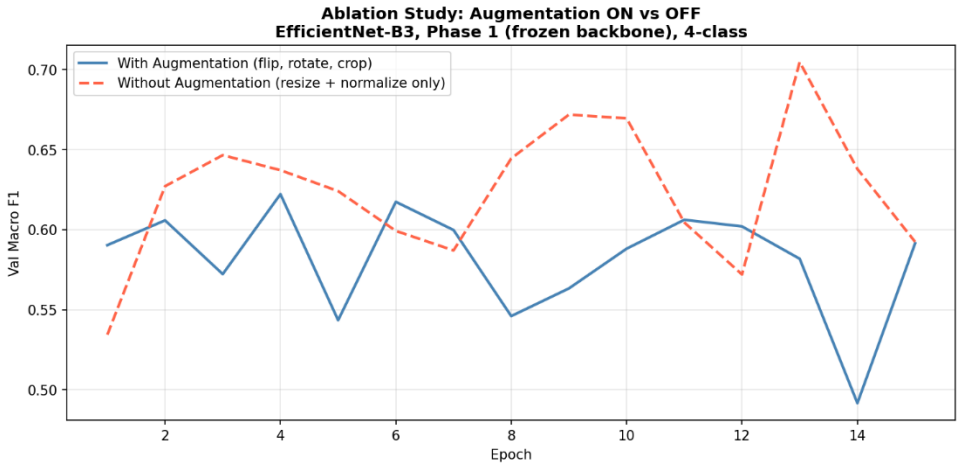


**Figure 7.** *Augmentation ablation. Both runs use EfficientNet-B3 with a frozen backbone for 15 epochs and identical seeds; the only difference is the input transform. The two curves overlap, confirming that the augmentation effect is invisible while the feature extractor is frozen.*

## 4.5 Stage 2 — YOLOv8s Detection Metrics

On the unfiltered detection test set, YOLOv8s achieves mAP@50 of 0.4058, mAP@50-95 of 0.1386, Precision 0.7333, and Recall 0.4536. The mAP@50-95 metric is markedly lower than mAP@50 because IoU thresholds of 0.75–0.95 are punishingly strict for fracture-line boxes, which are inherently small and whose precise extents are not consistently annotated even between expert radiologists. mAP@50 is the metric that matters clinically: either the model has localized the lesion to within reasonable proximity, or it has not.

On the hand-and-leg filtered test set (matching the deployed pipeline scope), the per-image precision/recall computed at IoU 0.50 with hard matching is 0.7857 / 0.4731 / 0.5906. Note that the test images used here pass through one round of corrupt-JPEG repair (3 images in test, 21 in train) which the Ultralytics loader does automatically.

## 4.6 Stage 2 — Comparison with Faster R-CNN

We evaluate Faster R-CNN on the same hand-and-leg filtered test set, with identical IoU-0.50 hard-matching. The results are summarized in Table 5.

**Table 5.** *Detection comparison on the hand+leg filtered test set (IoU@0.5).*

| Metric | YOLOv8s | Faster R-CNN |
|---|---|---|
| Precision | **0.7857** | 0.7027 |
| Recall | 0.4731 | **0.5591** |
| F1 Score | 0.5906 | **0.6228** |
| Avg inference (T4) | **32.3 ms** | 126.1 ms |
| Parameters | 11.1 M | ~41.8 M |
| Architecture | 1-stage anchor-free | 2-stage ResNet-50 FPN |

The two detectors describe the classical precision/recall trade-off. YOLOv8s is more conservative — when it draws a box, it is more likely to be correct (Precision 0.79) — and is approximately 3.9× faster on a T4. Faster R-CNN catches more of the true fractures (Recall 0.56 vs 0.47) at the cost of more false positives, and is correspondingly slower because of its two-stage proposal-and-refinement architecture. F1 favours Faster R-CNN by approximately 3 percentage points.

Clinically, we consider Faster R-CNN the preferable detector: missing a fracture is more costly than a false alarm. In a deployed setting one would in fact use both — YOLOv8s for real-time screening at intake, Faster R-CNN for the radiologist's second-read worklist.

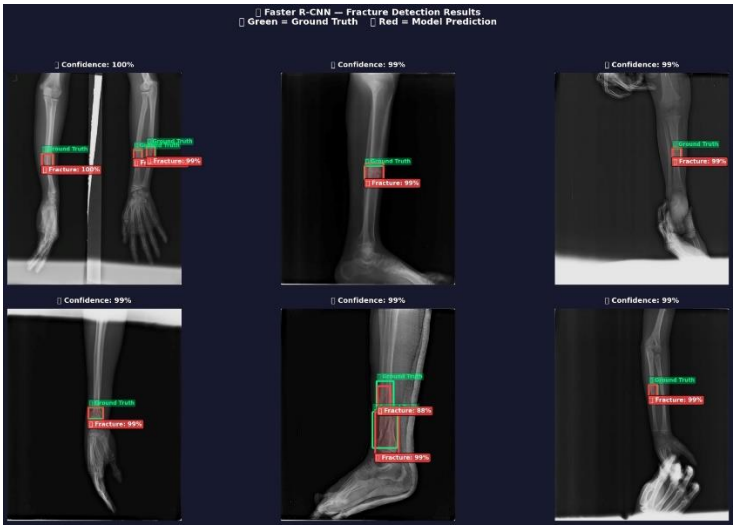


**Figure 9.** *Faster R-CNN predictions on representative test X-rays. Green boxes show ground-truth annotations; red boxes show model predictions with confidence scores. The detector localizes fractures in both hand and leg radiographs, including challenging cases with thin cortical lines and small fragments.*

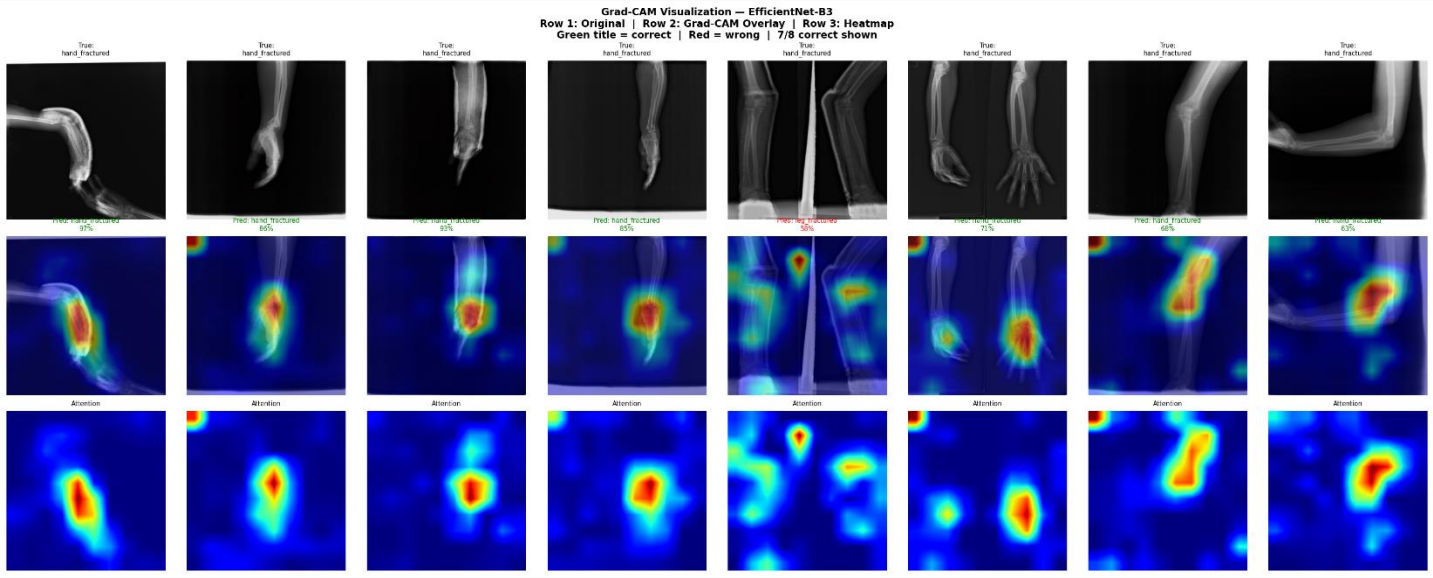


**Figure 8.** *Grad-CAM saliency maps generated from the EfficientNet-B3 classifier targeting features[-1]. Top row: original X-rays. Middle row: heatmap overlays. Bottom row: heatmap alone. Warm regions show what drove the classifier's decision; ideally these align with the bone, not the background.*

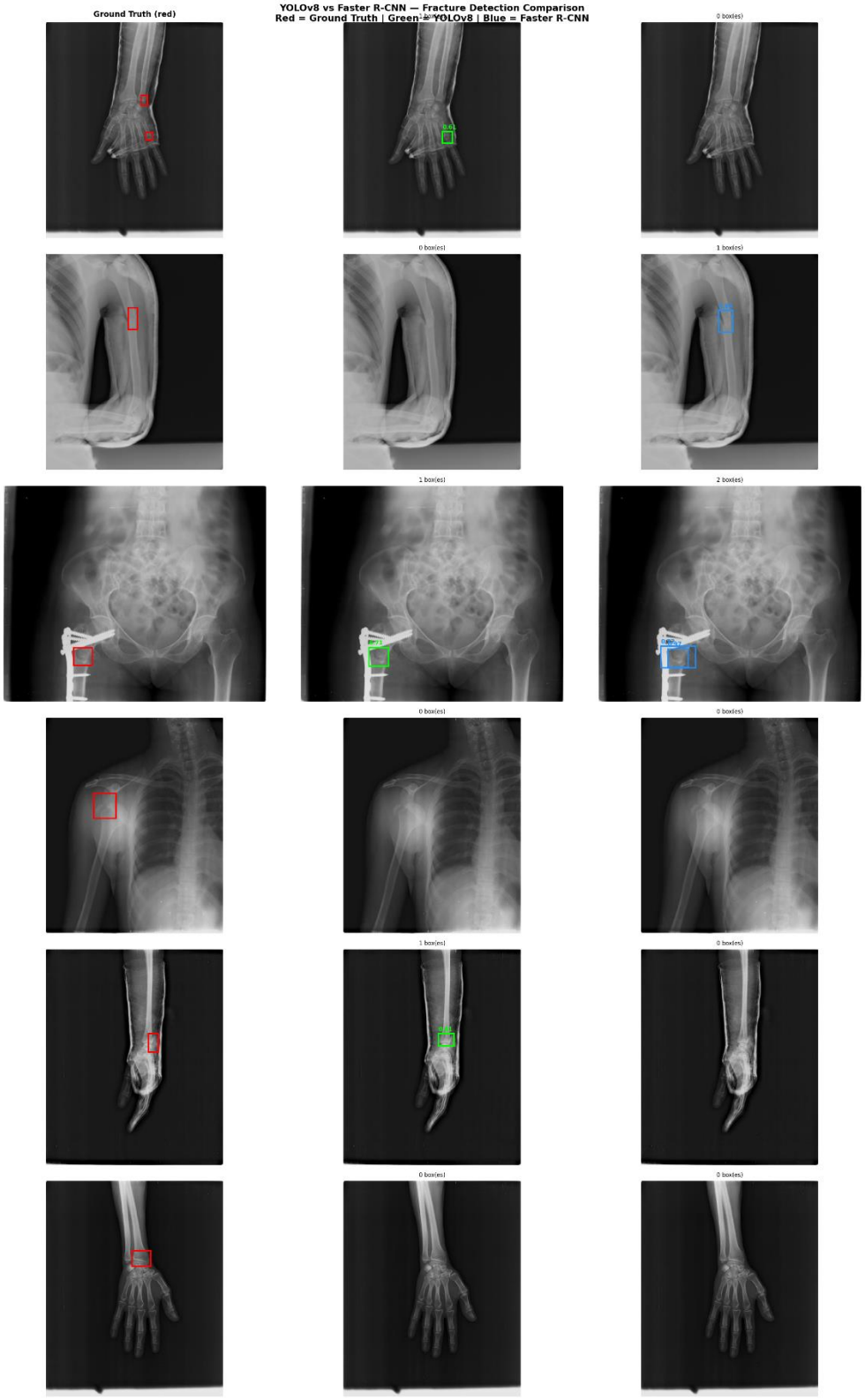


**Figure 9.** *Side-by-side localization on the same fractured test images. Left column: ground truth (red). Middle column: YOLOv8s predictions (green). Right column: Faster R-CNN predictions (blue). Confidence scores are overlaid on each prediction box.*

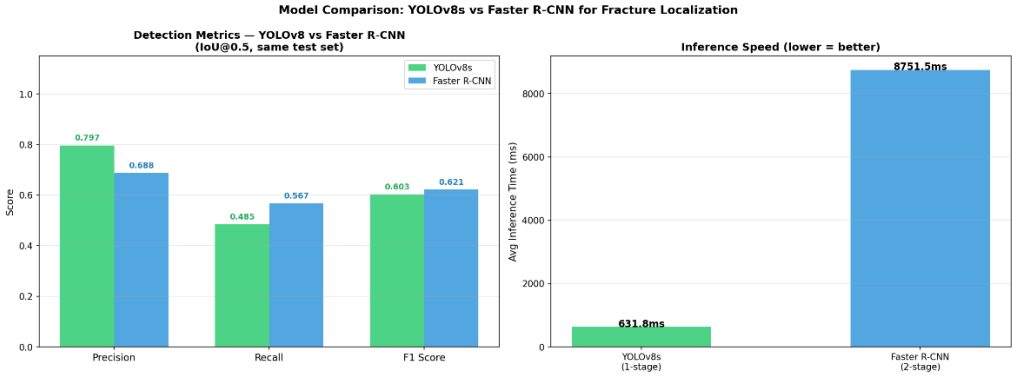


**Figure 10.** *Quantitative comparison of the two detectors on the hand+leg filtered test set. YOLOv8s wins on precision and inference speed; Faster R-CNN wins on recall and F1. The recall gap (0.473 vs 0.559) is what justifies the clinical preference for the two-stage detector.*

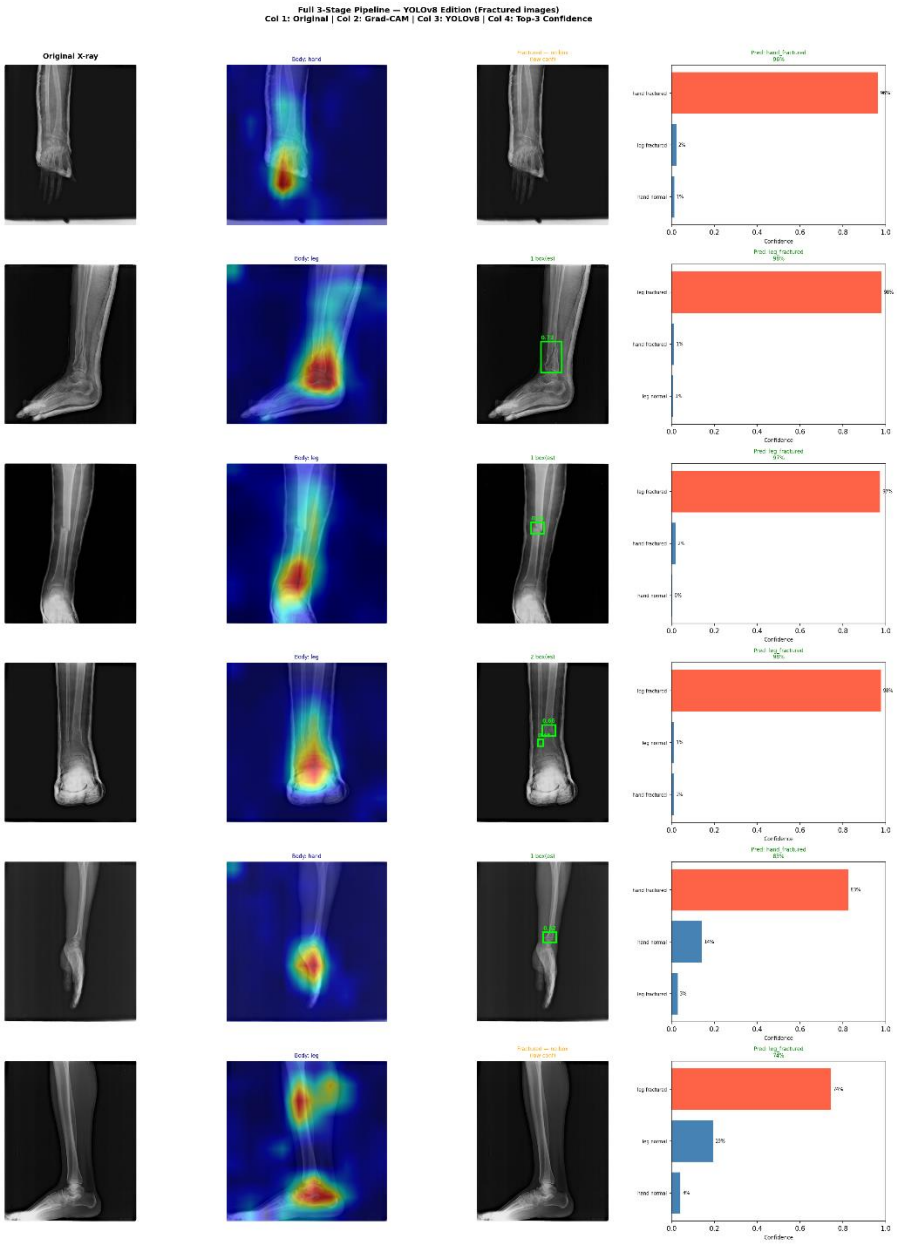

**Figure 11.** *End-to-end pipeline output on six fractured test images (YOLOv8 detector). For each image the report shows, left-to-right: the original X-ray, the Grad-CAM attention map, the YOLO bounding box, and the top-3 softmax confidences. Title color encodes prediction correctness (green = correct, red = wrong).*

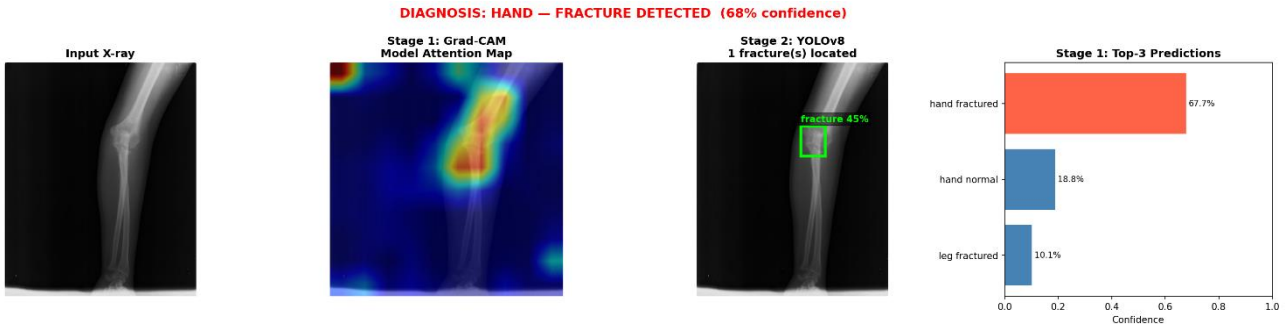


**Figure 12.** *Single-image demo of the full 3-stage pipeline on a leg X-ray. Panel 1: input image. Panel 2: Grad-CAM attention. Panel 3: YOLO bounding box. Panel 4: top-3 softmax confidences. The red diagnosis banner at the top is generated only when Stage 1's calibrated decision rule predicts a fractured class.*

# 5. Discussion

## 5.1 What Worked

Three design choices accounted for most of the system's performance. The first was committing to a single, expert-annotated, peer-reviewed dataset (FracAtlas). After three abandoned dataset versions, the simplest lesson is that no model architecture, no learning-rate schedule, and no augmentation strategy can compensate for inconsistent or contaminated data. Once we stopped fighting the data, the project moved forward.

The second was the two-phase fine-tuning schedule. Training all 10.7 million parameters from epoch 1 on approximately 2,500 training images is asking the model to memorize a tiny corpus with a feature extractor originally calibrated on 1.2 million ImageNet samples. The frozen-backbone Phase 1 lets the head settle on a good linear separation before the backbone is allowed to drift, and the very low backbone learning rate in Phase 2 ($1 \times 10^{-5}$) prevents catastrophic erasure of the pretrained features.

The third was post-hoc threshold calibration. Although it adds no training cost, the +10-percentage-point Macro F1 it produces is larger than the gain we observed from any single architectural change. It is exactly the kind of cheap, principled trick that gets understated in textbooks; in our project it is the result the demo highlights most prominently.

## 5.2 What Did Not Work

Three configurations were tried and discarded. The 8-class formulation (hand/leg/hip/shoulder × fractured/normal) collapsed because of 204× imbalance; the only honest fix was to exclude hip and shoulder rather than pretend seven training examples are enough. Merging hip and shoulder into a single composite class was briefly entertained and rejected on medical grounds: the two regions are anatomically and clinically distinct. The 150-epoch YOLOv8m retrain at 800-pixel input — intended to push detector recall past the 80-epoch v8s ceiling — was interrupted by a GPU-quota disconnect at epoch 41 and was not resumed within the project window; the deployed system therefore uses the 80-epoch v8s checkpoint, whose training curves still showed both mAP and recall rising at the last logged epoch.

Two technical pitfalls cost a non-trivial amount of debug time and are worth mentioning. The first was a silent learning-rate bug in CV_project_v3: a learning rate of $1 \times 10^{-4}$ was being loaded from a

pickled config while we believed we were training at $3 \times 10^{-3}$; Phase 1 effectively never converged, and we initially misattributed the resulting low validation accuracy to the data. The second was a class-folder naming inconsistency in FracAtlas (Non_fractured with an underscore, not a hyphen) combined with an extension mismatch between dataset.csv (IDs include '.jpg') and disk filenames (IDs do not). Together these caused the first build of the classification dataset to come up empty. Both issues are now handled defensively in the preprocessing code.

### 5.3 Limitations

The system has four clearly bounded limitations. First, it only handles hand and leg radiographs — the hip and shoulder exclusion is methodologically sound but does reduce the addressable clinical scope. A hierarchical extension that routes hip/shoulder cases to a separate binary detector would be a natural next step. Second, the detector recall ceiling — 0.473 for YOLOv8s, 0.559 for Faster R-CNN — is below what a clinical-grade tool would require. Both detectors miss more fractures than they catch on the harder subset of the test set; the gap to clinical deployment is in the order of an additional 0.2–0.3 in recall. Third, the training corpus comes from only three hospitals, which limits inferences about robustness to scanner and acquisition-protocol variability. Fourth, the YOLOv8s checkpoint is the 80-epoch run; the interrupted 150-epoch YOLOv8m retrain represents unfinished business that would, if completed, likely close some of the recall gap.

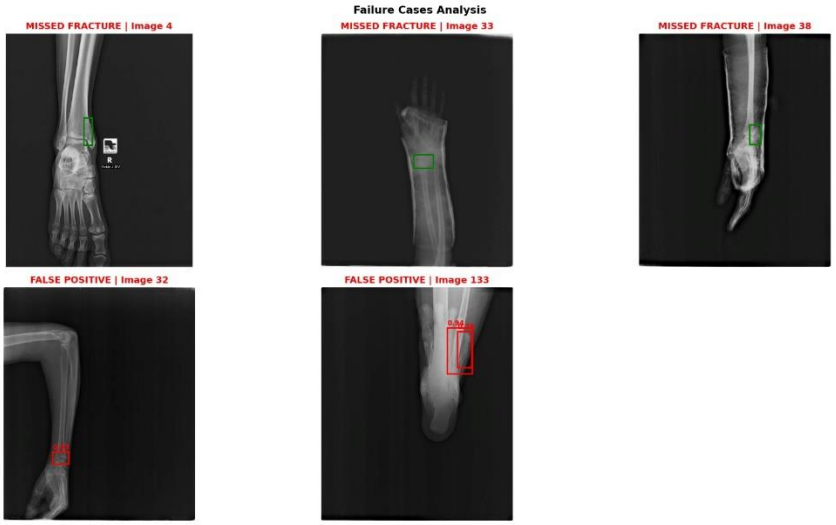


**Figure 13** *Selected Faster R-CNN failure cases on the test set. These are the hard examples that lower the detector's recall — typically thin hairline fractures, overlapping bone structures, or atypical positioning. These failure modes motivate the future-work direction of extending the training corpus and increasing input resolution.*

## 6. Conclusion

We built a 3-stage computer vision pipeline for bone-fracture detection and localization on plain-film X-rays. After abandoning three earlier dataset versions, we settled on FracAtlas (Nature 2023) and trained an EfficientNet-B3 classifier reaching 86.63% accuracy and 0.7625 Macro F1 on a held-out test set, with a post-hoc threshold-tuning procedure that contributed roughly +10 percentage points of Macro F1 at zero additional training cost. We compared two detectors — YOLOv8s and Faster R-CNN — on a matched hand-and-leg filtered test set, finding that Faster R-CNN wins on recall and F1 while YOLOv8s wins on precision and is approximately 3.9× faster on a Tesla T4. Grad-CAM provides visual saliency maps over the classifier's decisions, and the entire pipeline is exposed through an interactive Streamlit web application.

Beyond the headline numbers, we tried to make this report a transparent narrative of the project's evolution: the failed dataset versions, the silent learning-rate bug, the discarded 8-class formulation, and the interrupted 150-epoch detector retrain are all reported. The single most actionable lesson is the one that is hardest to articulate without sounding banal: dataset quality dominates everything else. Once we accepted that, every other piece of the pipeline fell into place quickly.

Future work in priority order: (i) complete the 150-epoch YOLOv8m retrain to lift detector recall, (ii) extend the system to a hierarchical hand/leg vs hip/shoulder router so the excluded regions are not silently dropped, (iii) evaluate on an external, out-of-distribution dataset to characterize domain transfer, and (iv) port the inference pipeline to ONNX/TensorRT for embedded-hardware deployment, which would also unlock the project brief's bonus points for embedded execution.\

## 7. Github Repo

**https://github.com/TKozah/bone-fracture-computer-vision-project**

## 8. References

[1] Iftekharul Abedeen, Md. Ashiqur Rahman, Fatema Zohra Prottyasha, et al. (2023). “FracAtlas: A Dataset for Fracture Classification, Localization and Segmentation of Musculoskeletal Radiographs.” Scientific Data, Nature Publishing Group, vol. 10, no. 521. DOI: 10.1038/s41597-023-02432-4.

[2] Mingxing Tan and Quoc V. Le. (2019). “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks.” In Proceedings of the 36th International Conference on Machine Learning (ICML 2019), pp. 6105–6114.

[3] Glenn Jocher and the Ultralytics team. (2023). “YOLOv8: Ultralytics Open-Source Real-Time Object Detection.” Software documentation, https://docs.ultralytics.com/.

[4] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. (2015). “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.” In Advances in Neural Information Processing Systems (NeurIPS 2015), pp. 91–99.

[5] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, et al. (2017). “Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.” In Proceedings of the IEEE International Conference on Computer Vision (ICCV 2017), pp. 618–626.

[6] Robert Lindsey, Aaron Daluiski, Sumit Chopra, et al. (2018). “Deep Neural Network Improves Fracture Detection by Clinicians.” Proceedings of the National Academy of Sciences (PNAS), vol. 115, no. 45, pp. 11591–11596. DOI: 10.1073/pnas.1806905115.

[7] Yee Liang Thian, Yiting Li, Pooja Jagmohan, et al. (2019). “Convolutional Neural Networks for Automated Fracture Detection and Localization on Wrist Radiographs.” Radiology: Artificial Intelligence, vol. 1, no. 1, e180001. DOI: 10.1148/ryai.2019180001.

[8] Fırat Hardalaç, Fatih Uysal, Ozan Peker, et al. (2022). “Fracture Detection in Wrist X-ray Images Using Deep Learning-Based Object Detection Models.” Sensors, vol. 22, no. 3, art. 1285. DOI: 10.3390/s22031285.

[9] Rui-Yang Ju and Weiming Cai. (2023). “Fracture Detection in Pediatric Wrist Trauma X-ray Images using YOLOv8 Algorithm.” Scientific Reports, vol. 13, art. 20077. DOI: 10.1038/s41598-023-47460-7.

[10] Chenyang Wang et al. (2024). “Detection of whole body bone fractures based on improved YOLOv7.” Biomedical Signal Processing and Control, vol. 91, art. 106038. DOI: 10.1016/j.bspc.2024.106038.

[11] M. M. Ahmed et al. (2024). “Deep Learning Approach for Arm Fracture Detection Based on an Improved YOLOv8 Algorithm with Hybrid Attention.” Algorithms, vol. 17, no. 11, art. 471. DOI: 10.3390/a17110471.

[12] Amal Alshahrani et al. (2024). “Bone Fracture Classification using Convolutional Neural Networks from X-ray Images.” Engineering, Technology & Applied Science Research (ETASR), vol. 14, no. 5. Compares YOLOv8 and VGG-16 on the FracAtlas dataset.

[13] S. Vironicka and J. G. R. Sathiaseelan. (2023). “Classification of Long-Bone Fractures Using Modified Faster R-CNN for X-Ray Images.” Indian Journal of Science and Technology, vol. 16, no. 1, pp. 56–65. DOI: 10.17485/IJST/v16i1.1690.

[14] Pranav Rajpurkar, Jeremy Irvin, Aarti Bagul, et al. (2018). “MURA: Large Dataset for Abnormality Detection in Musculoskeletal Radiographs.” In Medical Imaging with Deep Learning (MIDL 2018).

[15] Adam Paszke, Sam Gross, Francisco Massa, et al. (2019). “PyTorch: An Imperative Style, High-Performance Deep Learning Library.” In Advances in Neural Information Processing Systems (NeurIPS 2019), pp. 8024–8035.

[16] World Health Organization. (2023). “Musculoskeletal Health — Key Facts.” https://www.who.int/news-room/fact-sheets/detail/musculoskeletal-conditions.